Informe de Diseño y Análisis: Komikan Concurrente

Ramo: Sistemas Operativos | **Integrantes:** Bastián Ampuero y Guliano Bardi

1. Diseño del Esquema de Sincronización

Para organizar los hilos del torneo dividimos la sincronización en tres partes separadas: el tablero de juego, el registro de jugadas del juez y el control de mesas en el servidor. Para esto se ocuparon herramientas como (`pthread_mutex_t`, `pthread_cond_t` y `sem_t`) junto con variables atómicas (`std::atomic<bool>`) para avisar cuando una partida o el servidor deben detenerse.

1.1. Protección del tablero (`Tablero::candado`)

Cada mesa tiene su propio `struct Tablero` protegido por un solo mutex llamado `Tablero::candado`. Todas las funciones que leen o cambian las casillas (`tablero_mover_perro`, `tablero_mover_puma`, `tablero_saltar_puma`, `tablero_puma_acorralado` y `tablero_a_texto`) hacen `pthread_mutex_lock` al principio y `pthread_mutex_unlock` antes de salir.

- **Movimientos al mismo tiempo:** Los 13 hilos de las fichas (el puma y los 12 perros) juegan sin turnos y duermen un tiempo al azar entre `retardo_min_ms` y `retardo_max_ms` antes de intentar moverse. Si dos fichas intentan ir a la misma casilla vacía al mismo tiempo, el mutex deja pasar primero a una sola: esa ficha ve que la casilla está en `Casilla::Vacia`, se mueve y suelta el candado. Cuando entra la segunda ficha un instante después, ve que la casilla ya está ocupada, la función le retorna `false` y prueba con otra dirección.
- **Saltos del puma:** Cuando el puma come un perro con `tablero_saltar_puma`, tiene que revisar y cambiar tres casillas de una sola vez (donde parte, donde está el perro y donde cae), además de marcar `perro_vivo[idx] = false` y sumar uno a `perros_capturados`. Al usar un único candado para todo el tablero, nos aseguramos de que ningún perro se mueva a mitad del salto.

1.2. Comunicación entre las fichas y el juez (`candado_eventos` y `hay_evento`)

Para que el juez no gaste CPU preguntando todo el rato si hubo cambios y tampoco deje bloqueado el tablero mientras imprime por pantalla, hicimos una lista de eventos (`std::vector<std::string>eventos`) protegida por otro mutex (`candado_eventos`) y una variable de condición (`hay_evento`).

- Cuando una ficha logra moverse (ya habiendo soltado el candado del tablero), llama a `registrar_evento`. Ahí toma `candado_eventos`, guarda el texto de la jugada en el vector y despierta al juez con `pthread_cond_signal(&m.hay_evento)`.
- El juez pasa la mayor parte del tiempo dormido en `pthread_cond_wait`. Cuando una ficha lo despierta, saca todos los mensajes acumulados de una vez usando `lista.swap(m.eventos)`, suelta el candado rápido y recién ahí se pone a imprimir los tableros y revisar si alguien ganó o abandonó.

1.3. Límite de mesas en el servidor (`cupos_mesas` y `candado_mesas`)

En el servidor usamos el semáforo `cupos_mesas` (iniciado en M) para que nunca haya más de M mesas jugando al mismo tiempo.

- Cada vez que llegan dos clientes listos para jugar, el servidor hace `sem_wait(&servidor.cupos_mesas)`. Si las M mesas están llenas, el servidor se queda esperando ahí mismo sin gastar procesador hasta que termine alguna mesa y su juez llame al callback `servidor_mesa_terminada`, que hace `sem_post` y libera un cupo.
- Las listas de mesas (`mesas_activas` y `terminadas`) se cuidan con el mutex `candado_mesas`. Como el hilo del juez no puede hacerse `pthread_join` a sí mismo al terminar su partida, guarda la mesa en el vector `terminadas` para que después el servidor haga el `join` y libere la memoria con `mesa_destruir`.

2. Análisis de Riesgos de Concurrencia

2.1. Cómo evitamos el Deadlock

Para no tener bloqueos mutuos entre hilos, tuvimos en cuenta tres cosas:

1. **Nunca tomamos dos candados a la vez:** Ninguna función toma `Tablero::candado` y `candado_eventos` al mismo tiempo. Las fichas primero hacen su jugada en el tablero (toman y sueltan `Tablero::candado`) y recién después avisan al juez (toman y sueltan `candado_eventos`). Así es imposible que dos hilos se queden esperando cruzados.
2. **Parada limpia desde el servidor:** Cuando se aprieta `Ctrl+C`, `servidor_detener` recorre las mesas teniendo tomado `candado_mesas` y llama a `mesa_pedir_parada`. Adentro de esa función solo se cambia la variable `detener = true` y se despierta al juez con `candado_eventos`, sin volver a pedir `candado_mesas`.
3. **Soltar candados antes de los `join`:** Cuando termina la partida, el juez marca `m.detener = true` y suelta cualquier mutex antes de hacer `pthread_join` a las 13 fichas. Así, las fichas que estaban durmiendo en su `usleep` pueden despertar, salir de su ciclo `while` sin quedarse trabadas y terminar bien.

2.2. Cómo evitamos el Livelock y Starvation

- ***Livelock*:** Si dos fichas intentaran moverse sin pausa o probaran las direcciones siempre en el mismo orden (por ejemplo, siempre hacia arriba primero), podrían chocar una y otra vez sin avanzar. Para evitarlo, cada ficha duerme un tiempo aleatorio distinto entre `retardo_min_ms` y `retardo_max_ms` fuera del candado, y además antes de cada turno desordena al azar el arreglo `orden[4] = {0,1,2,3}` para ir variando por dónde intenta moverse.
- ***Starvation*:** Las operaciones dentro de `Tablero::candado` son muy cortas (solo mirar casillas en una matriz de 7×5 y cambiar un par de valores), lo que toma microsegundos en comparación a los 50 o 300 ms que duerme cada ficha en `usleep`. Como el candado pasa casi todo el tiempo libre, todas las fichas alcanzan a entrar sin quedarse esperando indefinidamente.

2.3. Condiciones de Carrera y Manejo de Ctrl+C

- **Números aleatorios por hilo:** En vez de usar `rand()` normal (que comparte estado entre todos los hilos y puede dar problemas de carrera), le pasamos a cada ficha su propia semilla en `ArgumentosFicha` y usamos `rand_r(&info.semilla)`.
- **Señal `SIGINT`:** Cuando el usuario presiona `Ctrl+C`, la función `manejar_sigint` solo cambia la variable atómica `aceptando_clientes = false`, porque meter mutexes o `cout` dentro de un manejador de señales no es seguro. También configuramos `sigaction` sin `SA_RESTART` para que, si el servidor estaba detenido en `sem_wait` esperando una mesa libre, se destrabe de inmediato y cierre todas las partidas activas.

3. Evaluación del Diseño

Mientras programábamos la tarea, tuvimos que equilibrar la complejidad de los candados con el rendimiento del juego:

- **Un solo mutex por tablero:** Al principio pensamos en poner un mutex por cada casilla de la matriz de 7×5, pero cuando el puma salta tiene que usar tres casillas al mismo tiempo (donde parte, el perro que se come y donde cae), y bloquear tres candados seguidos nos iba a dar problemas de *deadlock*. Como mover una ficha adentro del mutex es solo cambiar un par de variables en memoria y las fichas duermen entre 50 y 300 ms afuera, dejar un solo `Tablero::candado` nos simplificó el código y en las pruebas las jugadas salen al instante.
- **Separar la impresión del tablero:** Al principio probamos guardando solo el último movimiento en un string, pero como las fichas se mueven rápido, el juez se saltaba jugadas al imprimir. Por eso lo cambiamos al vector `eventos` con `swap` y su propio `candado_eventos`: así las fichas dejan su mensaje y siguen jugando de inmediato sin quedarse esperando el `cout` de la consola, manteniendo un buen *throughput*.
- **Mesas independientes:** Como cada `Mesa` tiene sus propios mutexes y variables de condición adentro de su struct, las M mesas activas juegan en paralelo de verdad, sin que una partida atrase a las otras.